

# Lattice Boltzmann method: 2D flow past a cylinder

## 1 Serial Profiling

I first profiled the serial solver on the default validation case  $800 \times 400$ ,  $Re = 100$ ,  $u_{in} = 0.05$ , and  $T = 60000$ , with snapshot and probe output disabled:

```
./lbm nx=800 ny=400 re=100 u_in=0.05 steps=60000 every=0 probe=/dev/null
```

The code was compiled with `-pg` and analysed using `gprof`. The flat profile showed that the runtime is dominated by the main lattice sweeps: `LBM::collide()` accounts for 71.88% of the sampled time, `LBM::stream()` for 25.82%, and `LBM::bounce_back()` for 1.98%. Since the MPI version parallelizes all three of these routines over the owned rows, I use the profiled parallel fraction

$$f = 0.7188 + 0.2582 + 0.0198 = 0.9968$$

for the Amdahl and Gustafson reference models.

## 2 MPI Parallelization

### 2.1 Design

The MPI version uses a one-dimensional row-block decomposition. Each rank owns a contiguous set of global rows plus two ghost rows, one above and one below. Since D2Q9 streaming only needs neighbouring cells, communication is limited to the upper and lower neighbouring ranks. The distribution functions keep the original structure-of-arrays layout, so the serial collision, bounce-back, streaming, inlet, and outlet routines require only local modifications.

### 2.2 Communication and Overlap Strategy

Each time step first performs BGK collision and bounce-back on the owned rows. The solver then starts a non-blocking halo exchange using `MPI_Irecv` and `MPI_Isend`. While communication is in flight, each rank streams its interior rows; after `MPI_Waitall`, it streams the boundary rows using the received ghost data. For pull streaming with row decomposition, only the three populations crossing each horizontal boundary are exchanged, reducing each halo message from  $9n_x$  to  $3n_x$  doubles.

### 2.3 Strong Scaling Results and Amdahl Comparison

Strong scaling was evaluated for three fixed global problem sizes,  $800 \times 400$ ,  $1200 \times 600$ , and  $1600 \times 800$ , all with  $T = 60000$ . Output was disabled in all timing runs using `every=0` and `probe=/dev/null`,

so the reported times measure the MPI solver rather than snapshot writing or probe output. For each problem size, speedup is computed relative to the corresponding  $p = 1$  MPI baseline on the same global grid.

The serial profile shows that the main lattice sweeps parallelized in the MPI solver, namely collision, streaming, and bounce-back, account for

$$f = 0.7188 + 0.2582 + 0.0198 = 0.9968$$

of the serial runtime. I therefore use the simple Amdahl model

$$S_{\text{Amdahl}}(p) = \frac{1}{(1 - f) + f/p}, \quad f = 0.9968,$$

as a first-order reference. With this value,  $S_{\text{Amdahl}}(256) \approx 140.97$ .

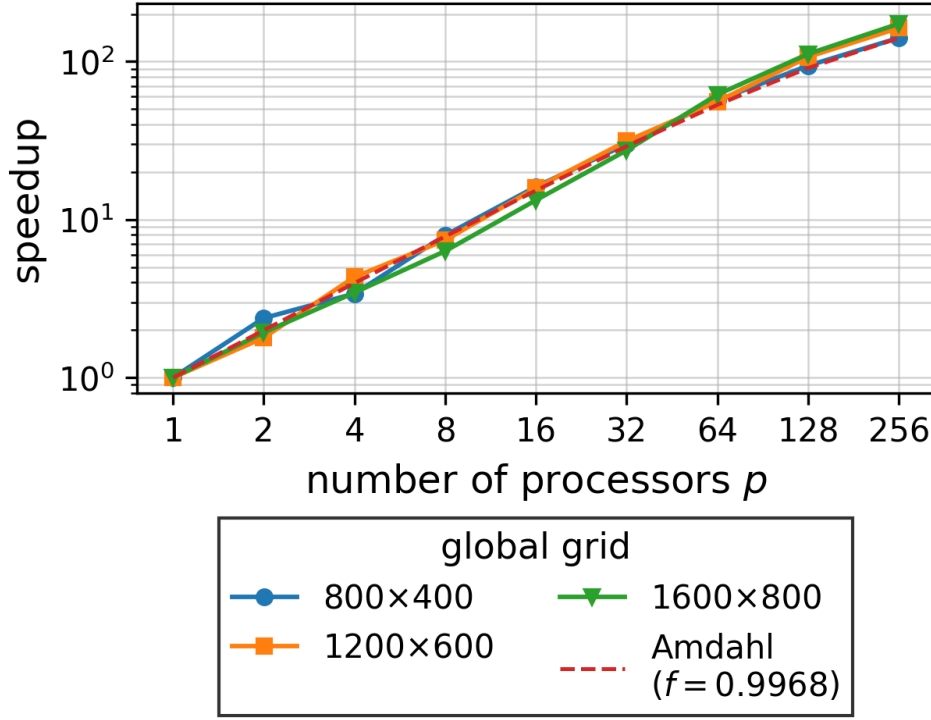


Figure 1: MPI strong scaling for three fixed global problem sizes, compared with a simple Amdahl prediction using the profiled parallel fraction  $f = 0.9968$ .

Figure 1 compares the measured strong-scaling speedup against this prediction. At  $p = 256$ , the measured speedups are 141.31, 163.22, and 173.64 for the  $800 \times 400$ ,  $1200 \times 600$ , and  $1600 \times 800$  cases respectively. The corresponding throughputs are 2848.78, 3658.97, and 3998.36 MLUPS, compared with the  $p = 1$  baselines of 20.16, 22.42, and 23.03 MLUPS. The  $800 \times 400$  case is close to the Amdahl estimate, while the two larger grids exceed it.

The speedups above the simple Amdahl curve are likely due to memory-hierarchy effects and differences between the profiled serial run and the MPI  $p = 1$  baseline. Splitting the domain gives each rank a smaller local block, which can improve cache locality, so the Amdahl curve is used as a reference rather than a strict bound.

At high process counts, communication grows relative to local computation. At  $p = 256$ , the communication fraction is 70.8% for the  $800 \times 400$  case, compared with 51.8% and 40.6% for the  $1200 \times 600$  and  $1600 \times 800$  cases. This shows that the one-dimensional row decomposition is ultimately limited by the surface-to-volume ratio of the local subdomains.

## 2.4 Weak Scaling Results and Gustafson Comparison

Figure 2 shows weak scaling with a fixed local height of 400 rows per rank. The global problem size therefore grows from  $800 \times 400$  at  $p = 1$  to  $800 \times 102400$  at  $p = 256$ . Thus, ideal weak scaling would keep the wall time nearly constant as  $p$  increases.

For the Gustafson comparison I use the same profiled parallel fraction as in the Amdahl model. This gives

$$\alpha = 1 - f = 0.0032, \quad S_{\text{Gustafson}}(p) = p - \alpha(p - 1).$$

The corresponding Gustafson efficiency is  $S_{\text{Gustafson}}(p)/p$ , which remains close to 100%; for example, at  $p = 256$ , the reference efficiency is approximately 99.68%.

The measured weak scaling follows this ideal only at small rank counts. At  $p = 2$ , efficiency is still 95.4%, but at  $p = 256$  it drops to 49.7%, while the measured communication fraction rises to 47.4%. Thus, synchronization and halo exchange dominate at high rank counts, so the measured behaviour falls well below the Gustafson reference.

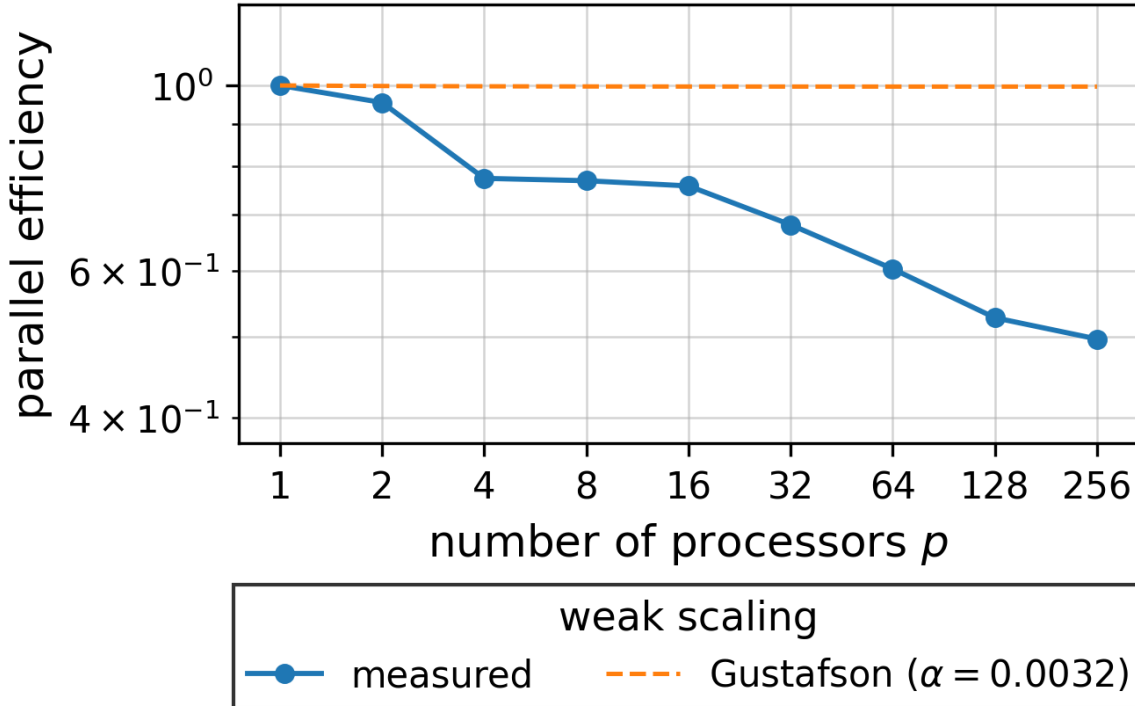


Figure 2: MPI weak scaling with fixed local height of 400 rows per rank, compared with the Gustafson efficiency reference using  $\alpha = 0.0032$ .

## 3 CUDA Acceleration

### 3.1 Design

The CUDA version moves the main lattice update to the GPU using a fused pull-streaming and BGK collision kernel. Each thread owns one destination cell, pulls the nine incoming D2Q9 populations from neighbouring cells into registers, computes  $\rho$  and  $\mathbf{u}$ , applies BGK collision, and writes the result to the next buffer. Thus, the intermediate post-collision state is not materialized in global memory.

This design reduces global memory traffic, gives each thread clear ownership of its output cell, and avoids a separate streaming kernel. The implementation uses double buffering: one device buffer stores the old distributions and the other stores the updated distributions, then the two pointers are swapped. Solid boundaries are handled by on-link bounce-back, and a small boundary kernel applies the inlet, outlet, and optional probe recording.

### 3.2 Memory Layout

The CUDA implementation uses a structure-of-arrays layout,

$$f[q(\textit{pitch} \cdot n_y) + y \cdot \textit{pitch} + x],$$

so neighbouring threads updating adjacent  $x$ -locations access contiguous memory for the same D2Q9 population. This improves coalescing compared with an array-of-structures layout. The row pitch can be padded as  $\textit{pitch} = \text{round\_up}(n_x, \textit{pitch\_align})$ , which improves alignment when  $n_x$  is not naturally aligned.

### 3.3 CUDA Cost Model and Roofline Estimate

The CUDA implementation is mainly memory-bandwidth limited. In the fused pull-streaming and collision kernel, each lattice update approximately reads nine double-precision populations and writes nine updated populations. This gives a lower-bound memory traffic estimate of

$$9 \text{ reads} + 9 \text{ writes} = 18 \text{ doubles} = 18 \times 8 = 144 \text{ bytes per lattice update.}$$

If the effective GPU memory bandwidth is  $B$  bytes/s, a simple bandwidth roofline estimate is

$$\text{MLUPS}_{\max} \approx \frac{B}{144} \times 10^{-6}.$$

Using a nominal V100 HBM2 bandwidth of 900 GB/s gives

$$\text{MLUPS}_{\max} \approx \frac{900 \times 10^9}{144} \times 10^{-6} \approx 6250 \text{ MLUPS.}$$

This is only a sanity check because the kernel also reads the solid mask and performs boundary checks and BGK arithmetic. The best measured result, 4906.88 MLUPS, is therefore plausible.

### 3.4 CUDA Performance Results

The CUDA performance study varies the global problem size, CUDA block shape, and row pitch alignment. All CUDA timings use `every=0` and `probe_every=0`, so snapshot output and probe recording are excluded.

The CPU baseline is the MPI implementation with  $p = 1$ , using the same grid, step count, and output-disabled setup. For  $800 \times 400$  and  $T = 60000$ , it reaches 23.87 MLUPS<sup>1</sup>, while the best CUDA run reaches 3979.15 MLUPS, giving a speedup of about  $166.7\times$ .

**Best configuration per problem size**

| Rank | Config                                 | Grid     | Block | Threads/block | Pitch align | Steps | Wall (s) | MLUPS          |
|------|----------------------------------------|----------|-------|---------------|-------------|-------|----------|----------------|
| 1    | cuda_nx400_ny200_bx32_by4_pa16_s60000  | 400x200  | 32x4  | 128           | 16          | 60000 | 1.69     | <b>2845.87</b> |
| 2    | cuda_nx800_ny400_bx64_by4_pa32_s60000  | 800x400  | 64x4  | 256           | 32          | 60000 | 4.83     | 3979.15        |
| 3    | cuda_nx1500_ny750_bx64_by4_pa64_s60000 | 1500x750 | 64x4  | 256           | 64          | 60000 | 13.76    | 4906.88        |
| 4    | cuda_nx1600_ny800_bx64_by4_pa32_s60000 | 1600x800 | 64x4  | 256           | 32          | 60000 | 15.67    | 4899.73        |

*Use this table for the problem-size discussion. Small grids are more affected by kernel launch overhead, while larger grids better expose memory-bandwidth limits.*

Figure 3: Best CUDA configuration for each problem size. Larger grids achieve higher throughput because they better amortize kernel-launch overhead and expose the memory-bandwidth-limited behaviour of the fused update.

Figure 4 focuses on the pitch-alignment sweep for the  $1500 \times 750$  grid. The best configuration uses a  $64 \times 4$  block with `pitch_align=64`, reaching 4906.88 MLUPS. The  $64 \times 4$  block is effective because it is wide in the  $x$ -direction, which matches the structure-of-arrays layout: neighbouring CUDA threads access adjacent  $x$ -locations for the same D2Q9 population. The pitch-alignment effect is modest overall, but padding helps slightly for this non-power-of-two grid width; `pitch_align=64` gives the best measured result on the  $1500 \times 750$  case.

**Block / pitch sweep for grid 1500x750**

| Rank | Config                                 | Grid     | Block | Threads/block | Pitch align | Steps | Wall (s) | MLUPS          |
|------|----------------------------------------|----------|-------|---------------|-------------|-------|----------|----------------|
| 1    | cuda_nx1500_ny750_bx64_by4_pa64_s60000 | 1500x750 | 64x4  | 256           | 64          | 60000 | 13.76    | <b>4906.88</b> |
| 2    | cuda_nx1500_ny750_bx64_by4_pa16_s60000 | 1500x750 | 64x4  | 256           | 16          | 60000 | 13.94    | 4843.51        |
| 3    | cuda_nx1500_ny750_bx64_by4_pa32_s60000 | 1500x750 | 64x4  | 256           | 32          | 60000 | 13.94    | 4841.47        |
| 4    | cuda_nx1500_ny750_bx64_by4_pa1_s60000  | 1500x750 | 64x4  | 256           | 1           | 60000 | 14.15    | 4771.73        |

Figure 4: CUDA pitch-alignment sweep for the best block shape on the  $1500 \times 750$  grid.

Overall, the CUDA results are consistent with the expected memory-bandwidth limited behaviour of the LBM update. Larger grids provide enough work to saturate the GPU more effectively, while block shapes with wider  $x$ -direction thread groups improve memory coalescing for the structure-of-arrays

<sup>1</sup>This uses the MPI weak-scaling  $p = 1$  run from the same output-disabled comparison setup. A separate strong-scaling  $p = 1$  run measured 20.16 MLUPS, but is not used for the CUDA speedup.

layout. Pitch padding has a smaller effect than problem size or block shape, but it can still provide a modest benefit for grid widths that are not naturally aligned.

## 4 Correctness Validation

Both parallel versions were validated on the fixed  $800 \times 400$ ,  $Re = 100$ ,  $T = 60000$  case using the transverse velocity probe and the provided `strouhal.py` script. For MPI, all process counts from  $p = 1$  to  $p = 256$  recovered  $St = 0.1600$ , showing that row decomposition and halo exchange preserve the result. Weak-scaling runs are used only for performance analysis because they change the global domain size.

For CUDA, nine launch and memory-layout configurations were tested, combining three block shapes,  $16 \times 16$ ,  $32 \times 8$ , and  $64 \times 4$ , with three pitch alignments, 1, 32, and 64. All nine runs recovered  $St = 0.1600$ . Although the fused pull-streaming/collision update with on-link bounce-back is not bitwise-identical to the serial collide–bounce\_back–stream sequence, it preserves the physical observable and reproduces the expected vortex street.

## AI Usage Statement

I used AI tools, mainly ChatGPT, to assist with generating initial drafts of the code, report, and presentation materials. AI was also used to help polish explanations.

All generated content was manually reviewed, modified, and verified by me. The final implementation, experiments, validation, performance analysis, and report interpretation were checked and finalized by me.